SCFF final

Muhammad Shayan ejaz

April 2026

1 Introduction

Below are the results of splitting SCFF in different scenarios. The main big problem I faced was that I was not using the `detach()` function properly in the original SCFF code. Because of that, `loss.backward()` would try to send gradients all the way back through Layer 0, which caused a lot of problems. In SCFF, that is not what we want. Each layer is supposed to learn more separately, so the gradients should stop at the right point instead of flowing through everything like normal backpropagation.

Another problem was with the concatenation of channels. Layer 0's convolution is defined with `ch_in = 3` because the input image is RGB. But at runtime, it was actually receiving a tensor of shape `[B, 6, H, W]`. This happened because two images were being joined together across the channel dimension to make a positive or negative pair. So instead of getting just one 3-channel image, the layer was getting two 3-channel images stuck together.

The solution was this:

```
if self.concat:
    lenchannel = x.size(1) // 2
    out = self.conv_layer(x[:, :3]) + self.conv_layer(x[:, 3:])
```

What this does is simple: it splits the 6 channels back into two halves of 3 channels each, runs the same convolution on both halves, and then adds the outputs together.

For positive pairs, where both halves are the same image or very similar, the outputs support each other, so the final response becomes stronger and the goodness becomes high. For negative pairs, where the halves are different images, the outputs do not match well and they cancel each other out more, so the goodness becomes lower. That is where the contrastive learning signal comes from. So in simple words, the network learns to give strong activations for related inputs and weaker activations for unrelated ones, even without using labels.

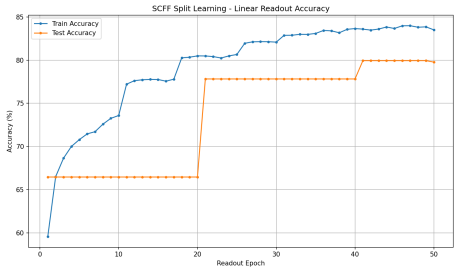

Figure 1: 1 client 1 server

```
Accuracy of the network on the 50000 train images: 80.482 %
Accuracy of the network on the 10000 Val images: 77.82 %
[21] loss: 0.001
Accuracy of the network on the 50000 train images: 83.59 %
Accuracy of the network on the 10000 Val images: 79.94 %
[41] loss: 0.001
Accuracy of the network on the 50000 train images: 83.506 %
Accuracy of the network on the 10000 Val images: 79.77 %
[50] loss: 0.001
Accuracy of the network on the 10000 Val images: 79.77 %
[50] loss: 0.001
Plot saved to scff_split_accuracy.png

Final Results:
  Train accuracy: 92.19%
  Test accuracy:  79.77%
```

Figure 2: 1 Client 1 server

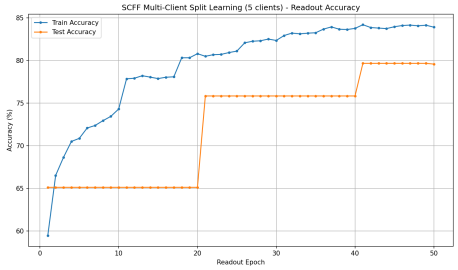

Figure 3: 5 clients 1 server

```
Accuracy of the network on the 50000 train images: 80.482 %
Accuracy of the network on the 10000 Val images: 77.82 %
[21] loss: 0.001
Accuracy of the network on the 50000 train images: 83.59 %
Accuracy of the network on the 10000 Val images: 79.94 %
[41] loss: 0.001
Accuracy of the network on the 50000 train images: 83.506 %
Accuracy of the network on the 10000 Val images: 79.77 %
[50] loss: 0.001
Accuracy of the network on the 10000 Val images: 79.77 %
[50] loss: 0.001
Plot saved to scff_split_accuracy.png

Final Results:
  Train accuracy: 92.19%
  Test accuracy:  79.77%
```

Figure 4: Configuration

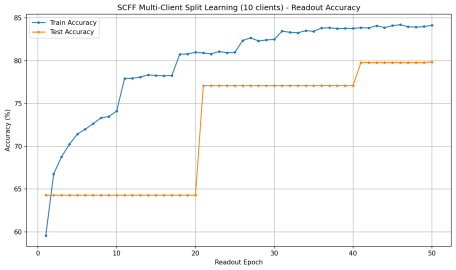

Figure 5: 10 clients 1 server